

Software Engineering - A.A. 24/25

"il disciplinare di esame":
guidelines for the self-assigned project work
with remarks on some recommended practices
(to be completed – vers.1.1 – Jan.25)



Enrico Vicario

Dipartimento di Ingegneria dell'Informazione

Laboratorio Tecnologie del Software

Università di Firenze

enrico.vicario@unifi.it, STLab.dinfo.unifi.it/vicario

■ Registration

- register for the exam on some date
- the exam can be discussed on that date or in subsequent days or weeks, agreed among the candidates and the teacher
- consider registering also for the integrated exam SWE+DB whenever DB was already passed

■ Presentation of the work

- send the relation in pdf attached to an email
- consider adding a link to github with the code
- do so at least 3 working days before the date of discussion of the exam, (if needed, you can then update the relation by sending a new mail in subsequent days)
- During the exam, bring a laptop with your development environment

■ Email

- Include the term [SWE] in the subject of all messages
- maintain the thread of messages, so as to create a history of interactions

- this is a self-assigned work
 - it is intended to have a concrete basis on which we can discuss, a kind of "free question" (domanda a scelta) to be discussed
 - for the discussion, the candidates must have full understanding and mastery on everything appears in the work
 - the work does not replace general understanding of course contents: the candidate should first go through and understand course contents, and after that the self-assigned work becomes a concrete occasion to bring them in action in the design and production of something
- review
 - you may want to ask a feedback on the assumed statement, and artifacts produced along the development
 - I'll try to answer, but this should not block your progress
 - feedback shall be asked on written concepts, trying to focus specific points of attention ... this is essential to permit concrete answers

- Generative AI
 - the use of Generative AI is welcome and promoted, not requested
 - document any usage you made: for which parts of the work, with which process of integration with your integrated development environment, ...
 - proficient usage of generative AI is a value, but this is not requested
 - quality and detail of this documentation is part of the value of the work

- do NOT generate UML from the code, this is normally of no use

- There are 3 different types of project work, accommodating different starting levels and aims of the candidate
 - Mode #2 is the common and most frequent case
 - Mode #1 simplifies the objective
 - Mode#3 makes the objective more ambitious

- Mode#2 is the common and most frequent case.
 - It consists in design and production of a fragment of application, referred to some application domain outlined in some statement, developed so as to give space to practices of SW illustrated in the course;
 - It can be conveniently developed by a single student, or by a team of 2 or 3 students, sometimes 4.
 - The way to identify the subject and develop the work of Mode#2 are specified in the sequel of this document

- Mode#1 simplifies the objective
 - it is an exercise of OO design, development and testing, typically centered on the combination of 2-3 design patterns motivated in some fictitious application scenario
 - (some examples were partially developed in the slides on Design Patterns, e.g. combining Composite, Observer, Adapter, Decorator, Strategy ...)
 - Mode#1 does not bring in action all the contents of the course, but it still permits to consolidate the ability to develop and test code using the Java language and its effective idioms, combining OO design principles and UML models
 - this is less ambitious than Mode#2 and attains a weaker proficiency, but it is a good choice when starting from a weak level of programming skill
 - It is normally (but not always) developed by a single candidate

- Mode#3 is a kind of special project work
 - it is intended for candidates with a good level in software development, who may be interested in applying this skill in the solution of some advanced computing problem
 - possible subjects include the development of quantitative analytics integrating probabilistic models, or in the experimentation of advanced practices of software engineering, ...
 - it is normally (but not always developed by a team of 2,3 students), and its contents are defined case by case depending on the specific interests and skills of candidates



- The rest of this document is about project works in Mode#2

- The report normally includes these sections
 - statement, architecture at glance, used technologies and tools
 - requirements analysis and functional design, use case diags, templates, mockups, page navigation diagram
 - design, architecture, package diagram, class diags for each package, details of specific significance
 - implementation details, responsibilities, snippets and discussion on elements of specific relevance, idioms, design patterns, architectural patterns, details on used technologies
 - tests, functional (black box, integration) and structural (white box, unit)
 - (see more in examples in the sequel)



- In the frontpage, mention course, academic year, authors, and teacher
- provide an index, and give sections titles to make the index self-explaining

- statement
 - this shall outline an application context where the candidate can identify requirements for the software that will be designed and produced
 - ... see later, by example

- Document the structure of packages and their dependencies with a diagram
 - provide a textual description of responsibilities of each package and the type of components that it includes (e.g. Controllers in the business logic, Entities in the domain model, DAOs in the ORM)

- List used technologies and tools
 - (e.g. JDBC, JavaFX, StarUML, Draw.IO, Balsamiq, ...Eclipse, Intelli-J, ... CoPilot, ...)

- Templates + Mockups + Navigation Diagram compose a triple
 - templates are mandatory
 - mockups and page navigation diagram are recommended
 - associate mockups pages in the page navigation diag (whenever these are developed)

- In templates, some fields have higher relevance
 - number, title, level, roles, normal and alternative flows, pre- and post-conditions
 - be able to distinguish whether the level is "user goal", "function", or "summary"
 - maintain consistency between titles and numbers appearing in use case diags and templates
 - steps in the normal flow and alternative flows can be associated with (available) mockups
 - normal flow and alternative flows can be associated with tests



- See later, by examples
- includes documentation of the database model



- See later, by examples

- Be able to distinguish whether tests are functional or structural (see later, by examples)
- consider using tests to surrogate user interface and persistence layer

- In the frontpage, mention course, candidates, teacher, Academic year



UNIVERSITÀ
DEGLI STUDI
FIRENZE

UNIVERSITÀ DEGLI STUDI DI FIRENZE
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

GreenHouse
Software per la gestione di serre e piante

Autori:



N° Matricola:



Corso principale:
Ingegneria del Software

Docente corso:
Enrico Vicario

- Give each title a meaning so as to obtain a self-explaining index

Indice

1	Introduzione	2
1.1	Statement	2
1.2	Architettura e pratiche utilizzate	2
2	Progettazione	3
2.1	Use Case Diagram	3
2.2	Use Case Template	4
2.3	Mock-ups	8
2.4	Navigation Diagram	11
2.5	Class Diagram	12
2.6	ER Diagram e modello relazionale	15
3	Implementazione	16
3.1	Domain Model	17
3.1.1	Utente	17
3.1.2	Pianta	17
3.1.3	Ordine	17
3.1.4	Impianto	18
3.1.5	Sensori	18
3.1.6	Attuatori	18
3.2	Business Logic	20
3.2.1	LoginClienteController	20
3.2.2	LoginPersonaleController	20
3.2.3	ClienteController	20
3.2.4	OperatoreController	20
3.2.5	AdminController	21
3.2.6	AdminExtraController	22
3.3	ORM (Object-Relational Mapping)	23
3.3.1	ConnectionManager	23
3.3.2	ClienteDAO	24
3.3.3	AdminDAO	26
3.3.4	OperatoreDAO	26
3.3.5	SensoreDAO, AttuatoreDAO e OperazioneDAO	26
3.3.6	SettoreDAO, PosizioneDAO	26
3.3.7	OrdineDAO e PiantaDAO	26
3.3.8	PosizionamentoDAO	29
3.4	Database	29
3.5	Interfaccia	30
4	Testing	31
4.1	Business Logic Test	31
4.1.1	LoginClienteControllerTest	31
4.1.2	LoginPersonaleControllerTest	33
4.1.3	ClienteControllerTest	33
4.1.4	OperatoreControllerTest	34
4.2	Domain Model Test	36
4.3	ORM Test	37

- Provide a clear yet synthetic statement of the application context and aim

1.1 Statement

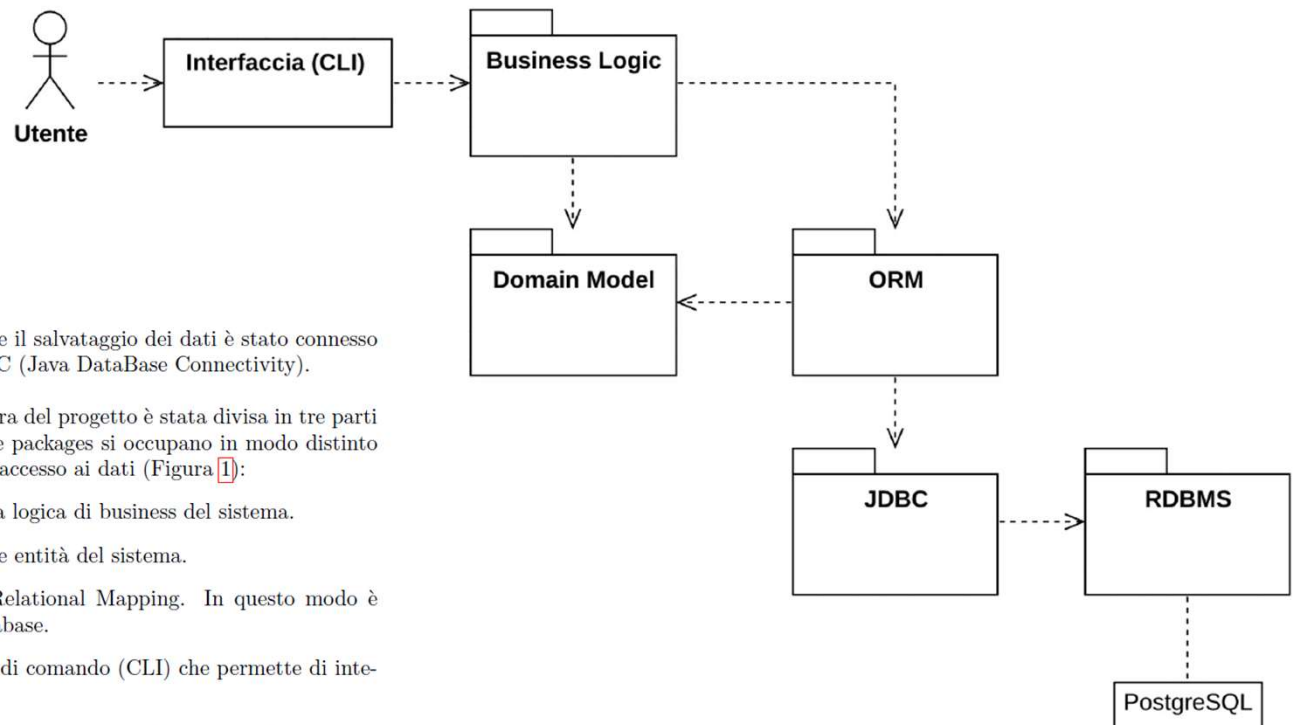
Il progetto modella un sistema di controllo e gestione di serre, e in particolare si occupa della cura e crescita delle piante in esse, al fine ultimo di venderle a clienti. I 3 spazi delle serre sono divisi in 4 settori in cui è presente un sistema di areazione, di illuminazione e controllo della temperatura, luce e umidità, e contengono 5 posizioni ciascuna nelle quali mettere le piante che saranno irrigate e monitorate.

La gestione delle piante è in gran parte automatizzata e possono essere monitorati sia i settori che le posizioni per ottenere un controllo *real-time* da parte dell'Admin.

Ci sono operazioni che dovranno essere effettuate manualmente come la cura e il posizionamento delle piante nelle posizioni designate, che saranno eseguite da operatori. Tale sistema è adottato da un'azienda che permette ai clienti di acquistare e commissionare un certo numero di piante con consegna prevista entro un certo periodo.

- TBD: Could be extended with a short outline of roles and actions, which, however, should not be extended as in use case diags and templates
- TBD this can be useful also when presenting the work

- Document the architecture with packages and their dependencies
- Accompanied by a textual description of responsibilities and constituency of each package



1.2 Archiettura e pratiche utilizzate

Il software è stato sviluppato in Java, mentre per la gestione e il salvataggio dei dati è stato connesso un database PostgreSQL ed è stata utilizzata la libreria JDBC (Java DataBase Connectivity).

Per mantenere una separazione delle responsabilità, la struttura del progetto è stata divisa in tre parti principali: Business Logic, Domain Model e ORM. Questi tre packages si occupano in modo distinto della logica di business, della rappresentazione dei dati e dell'accesso ai dati (Figura 1):

- **Business Logic:** contiene le classi che implementano la logica di business del sistema.
- **Domain Model:** contiene le classi che rappresentano le entità del sistema.
- **ORM:** contiene le classi che implementano l'Object-Relational Mapping. In questo modo è possibile rendere i dati persistenti e recuperarli dal database.

Per utilizzare il software è stata creata un'interfaccia da riga di comando (CLI) che permette di interagire con il sistema in modo semplice e intuitivo.

Gli Use Case Diagram e i Class Diagram seguono lo standard UML (Unified Modeling Language) e sono stati realizzati con il software StarUML. Infine, per la parte di testing è stato utilizzato JUnit.

Questa architettura prevede che l'utente esegua un'azione tramite l'interfaccia; questa richiama direttamente i metodi esposti nei controller della Business Logic; quest'ultima delega la creazione degli oggetti all'ORM che richiede i dati al database tramite JDBC e RDBMS; creati gli oggetti (definiti nel Domain Model), la Business Logic può usarli direttamente e portare a termine l'esecuzione della funzione desiderata. Vedi Figura 1.

- The example in this page is particularly complex, not to be assumed as a required level



- Characterize each use case with a number, title, brief description, level of abstraction, involved actors, pre- and post-condition, basic course and alternative courses
- Number alternative courses so as to identify their offspring from steps in the basic course
- Associate steps of the template with (available) mockups
- Consider associate basic and alternative courses with tests

- (multiple actors may occur sometimes. When they are too frequent, there may be a mis-understanding of their meaning)

Use Case #1	Accedi al Sistema (Sign-in)
Brief Description	L'utente accede al sistema tramite le proprie credenziali
Level	User Goal
Actors	Admin, Cliente, Operatore
Pre-Conditions	L'utente deve essere nella pagina dedicata al proprio accesso (Mockup #1)
Basic Flow	1) L'utente sceglie la propria pagina di Log In 2) L'utente inserisce le proprie credenziali 3) L'utente invia le proprie credenziali 4) Il sistema verifica le credenziali 5) Il sistema autentica l'utente (Test #3, #6 e #9)
Alternative Flow	3a) Se le credenziali sono errate o l'utente non è registrato, il sistema restituisce un messaggio di errore e permette di ritentare (Test #4, #5, #7, #8, #10, #11)
Post-Conditions	L'utente è autenticato nel sistema e ha accesso alle sue funzioni

■ idem

Use Case #3	Richiedi nuovo Ordine
Brief Description	Il cliente crea un nuovo Ordine
Level	User Goal
Actors	Cliente
Pre-Conditions	Il cliente deve aver fatto l'accesso e essere nella pagina "Ordini"
Basic Flow	1) Il cliente seleziona l'opzione per creare un nuovo ordine 2) Il cliente sceglie il tipo di pianta e la quantità desiderata (Mockup #3) 3) Il cliente sceglie se aggiungere altre piante o se concludere l'ordine 4) Il sistema registra il nuovo ordine (Test #12)
Alternative Flow	2a) Se il cliente inserisce valori non validi, il sistema mostra un messaggio di errore e richiede l'input 4a) Se l'ordine non può essere preso in carico, il sistema restituisce un messaggio (Test #13)
Post-Conditions	Il nuovo ordine è stato registrato nel sistema

Use Case #7	Check-Up Piante
Brief Description	L'Operatore verifica lo stato delle piante
Level	User Goal
Actors	Operatore
Pre-Conditions	L'Operatore deve essere sulla dashboard
Basic Flow	1) L'Operatore sceglie l'opzione "Check-Up Piante" 2) Per ogni Pianta del sistema viene eseguito un controllo che valuta se la pianta ha bisogno di cure (Test #17) 3) Se sono presenti piante che hanno bisogno, queste vengono curate (Test #19) 4) Il Check-Up è terminato 5) Il sistema salva l'operazione effettuata dall'operatore nel database
Alternative Flow	2a) Se non sono presenti piante su cui è possibile fare il check-up, il sistema mostra un errore (Test #18)
Post-Conditions	Le Piante sono in salute e possono continuare a crescere

- Provide snippets for code fragments with specific significance, and describe it

3.3.7 OrdineDAO e PiantaDAO

OrdineDAO e PiantaDAO, come le altre DAO, hanno i metodi per inserire, aggiornare e restituire gli oggetti associati, inoltre PiantaDAO ha metodi per eliminare elementi, usato quando l'ordine è espletato. Le due DAO sono direttamente collegate in quanto nel momento di richiesta di dati di un Ordine a OrdineDAO, il suo metodo `get` invoca il metodo `get` di un PiantaDAO in modo da inizializzare anche la lista di Piante, attributo della classe Ordine.

```
public ArrayList<Pianta> get(Map<String, Object> criteri) {
    StringBuilder query = new StringBuilder("SELECT * FROM \"Pianta\"");

    // Aggiungi condizioni se ci sono criteri
    if (criteri != null && !criteri.isEmpty()) {
        query.append(" WHERE ");
        for (String key : criteri.keySet()) {
            query.append(key).append(" = ? AND ");
        }
        query.setLength(query.length() - 5); // Rimuove l'ultimo AND
    }

    ArrayList<Pianta> piante = new ArrayList<>();

    try (PreparedStatement statement = connection.prepareStatement(query.toString())) {
        // Imposta i parametri se ci sono criteri
        if (criteri != null && !criteri.isEmpty()) {
            int paramIndex = 1;
            for (Object value : criteri.values()) {
                statement.setObject(paramIndex, value);
                paramIndex++;
            }
        }

        // Esegui la query e gestisci il ResultSet
        try (ResultSet resultSet = statement.executeQuery()) {
            while (resultSet.next()) {
                pianta.add(new Pianta(resultSet.getInt("id"),
                    resultSet.getString("tipo"), resultSet.getString("descrizione"), resultSet.getString("dataInizio"),
                    resultSet.getString("stato")));
            }
        }
    } catch (SQLException e) {
        System.err.println("Errore durante il recupero delle piante: " + e.getMessage());
    }

    return piante;
}
```

Snippet 7: metodo `get()` nella classe PiantaDAO

```
public ArrayList<Ordine> get(Map<String, Object> criteri) {
    StringBuilder query = new StringBuilder("SELECT * FROM \"Ordine\"");
    if (criteri != null && !criteri.isEmpty()) { // Aggiungi condizioni se ci sono criteri
        query.append(" WHERE ");
        for (String key : criteri.keySet()) {
            query.append(key).append(" = ? AND ");
        }
        query.setLength(query.length() - 5); // Rimuove l'ultimo AND
    }

    ArrayList<Ordine> ordini = new ArrayList<>();

    try (PreparedStatement statement = connection.prepareStatement(query.toString())) {
        // Imposta i parametri se ci sono criteri
        if (criteri != null && !criteri.isEmpty()) {
            int paramIndex = 1;
            for (Object value : criteri.values()) {
                statement.setObject(paramIndex++, value);
            }
        }

        PiantaDAO piantaDAO = new PiantaDAO();
        // Esegui la query e gestisci il ResultSet
        try (ResultSet resultSet = statement.executeQuery()) {
            while (resultSet.next()) {
                int idOrdine = resultSet.getInt("id");
                // chiama PiantaDAO per inizializzare le piante
                ArrayList<Pianta> piante = piantaDAO.get(Map.of("ordine", idOrdine));
                ordini.add(new Ordine(
                    idOrdine,
                    resultSet.getInt("cliente"),
                    piante,
                    resultSet.getString("stato"),
                    resultSet.getString("dataConsegna"),
                    resultSet.getDouble("totale")
                ));
            }
        }
    } catch (SQLException e) {
        System.err.println("Errore durante il recupero degli ordini: " + e.getMessage());
    }

    return ordini;
}
```

Snippet 8: metodo `get()` nella classe OrdineDAO

3.4 Database

Per mantenere coerenza tra i dati del sistema è stato utilizzato un database utilizzando PostgreSQL, con il quale è possibile interfacciarsi tramite le classi DAO. Tale database segue lo schema delle operazioni CRUD (CREATE, READ, UPDATE, DELETE).

Per inizializzare il database sono stati creati 3 file .sql:

- `reset.sql`: elimina tutte le tabelle del database (DROP TABLE);
- `schema.sql`: crea le tabelle del database definendo i vari attributi, chiavi e vincoli (CREATE TABLE);
- `default.sql`: inserisce valori nelle tabelle in modo da avere un con il programma (INSERT INTO).

Questi 3 file sono stati eseguiti, grazie ai metodi presenti in AdminC programma e sono utilizzati anche prima di ogni test (BeforeEach) precise richieste dai test.

```
CREATE TABLE IF NOT EXISTS "Cliente" (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(50),  
    cognome VARCHAR(50),  
    email VARCHAR(50),  
    password varchar(50)  
);  
  
CREATE TABLE IF NOT EXISTS "Ordine" (  
    id SERIAL PRIMARY KEY,  
    cliente INT,  
    dataConsegna VARCHAR(50),  
    piante VARCHAR(300),  
    totale DECIMAL(6,2),  
    stato VARCHAR(50),  
    FOREIGN KEY (cliente) REFERENCES "Cliente"(id)  
);  
  
CREATE TABLE IF NOT EXISTS "Pianta" (  
    id SERIAL PRIMARY KEY ,  
    tipo VARCHAR(50),  
    descrizione VARCHAR(200),  
    dataInizio VARCHAR(50),  
    stato VARCHAR(100),  
    costo DECIMAL(10, 2),  
    ordine int references "Ordine"(id)  
);
```

Snippet 10: creazione tabelle Cliente, Ordine e Pianta nel file `schema.sql`

- 1/3 - Perform functional tests (black box, integration)
by exercising methods exposed by Controllers in the business logic
(which surrogates the user interface)
- In so doing, refer to flows (basic and alternative) specified in templates

4.1 Business Logic Test

Molti di questi test presentano più variazioni, in quanto verificano sia il caso in cui le condizioni per una certa funzione siano favorevoli, quindi è atteso un esito positivo ("Success"), sia il caso in cui vengano riscontrati problemi e quindi ci si aspetta un esito negativo ("Fail").

I test sono suddivisi in più classi (rispettando la suddivisione che hanno le relative funzioni):
`LoginClienteControllerTest`, `LoginPersonaleControllerTest`, `ClienteControllerTest`,
`OperatoreControllerTest`.

■ Provide motivation and snippets of relevant tests

- Trace back tests to use cases and templates

4.1.1 LoginClienteControllerTest

Si effettuano i test delle funzioni utilizzate dal cliente per l'accesso e la registrazione al sistema quali:

- `setUp`: reimposta il database a uno stato iniziale default;
- `registrationTest_Success`: esegue il test del metodo `registraTi` di `loginClienteController`, utilizzata per la registrazione di un nuovo cliente (Use Case [#1](#));
- `registrationTest_Fail`: come `registrationTest_Success`, ma l'email con il quale si tenta di registrarsi appartiene a un cliente già presente nel database, quindi il test di aspetta un fallimento nella procedura (Use Case [#1](#));
- `loginTest_Success`: verifica la corretta funzionalità del metodo `accedi` che permette al cliente di accedere al sistema (Use Case [#2](#));
- `loginTest_Fail1` e `loginTest_Fail2`: verifica l'effettivo fallimento del login nel caso in cui la password o l'email non siano corrette (Use Case [#2](#));

```
LoginClienteController loginClienteController;  
String none, cognome, email, password;  
  
@Test  
public void registrationTest_Success() { //Nuovo cliente non presente nel db  
    nome = "Giuseppe";  
    cognome = "Verdi";  
    email = "giuseppe@email.it";  
    password = "123";  
    assertTrue(loginClienteController.registraTi(nome, cognome, email, password));  
}  
  
@Test  
public void registrationTest_Fail(){ //Cliente già presente nel db  
    nome = "Mario";  
    cognome = "Rossi";  
    email = "mario@email.it";  
    password = "123";  
    assertFalse(loginClienteController.registraTi(nome, cognome, email, password));  
}  
  
@Test  
public void loginTest_Success() { //Cliente presente nel db, credenziali corrette  
    email = "mario@email.it";  
    password = "123";  
    assertNotNull(loginClienteController.accedi(email, password));  
}  
  
@Test  
public void loginTest_Fail1() { //Cliente presente nel db, password errata  
    email = "mario@email.it";  
    password = "abc";  
    assertNull(loginClienteController.accedi(email, password));  
}  
  
@Test  
public void loginTest_Fail2() { //Cliente non presente nel db  
    email = "luigi.conti@gmail.com";  
    password = "1234567890";  
    assertNull(loginClienteController.accedi(email, password));  
}
```

- 2/3 - Perform unit tests (white box, structural)
for elements of specific relevance or criticality in the code

4.2 Domain Model Test

Per quanto riguarda il Domain Model è stato effettuato un solo test, in quanto sono presenti classi che fungono soltanto da entità, e generalmente sono passive, cioè prive di metodi e funzionalità.

I sensori però contengono un metodo `misura` che prende in ingresso l'orario e lo stato dell'attuatore associato a tale sensore e ritorna una misurazione del parametro fisico che gli compete. Questo simula l'influenza del tempo e degli attuatori sulla misurazione.

Nel caso in cui l'Irrigatore sia acceso (`testMisura_Irr_ON`), ci si aspetta un valore di umidità del terreno (misurato da `IgrometroTerra`) crescente nel tempo. Altrimenti (`testMisura_Irr_OFF`) il valore diminuisce (l'acqua viene assorbita dalla pianta). Il test verifica questo.

```
public class IgrometroTerraTest {  
    @Test  
    public void testMisura_Irr_ON(){  
        IgrometroTerra igrometroTerra = new IgrometroTerra(0, null, 10);  
        LocalDateTime lt = LocalDateTime.now();  
        float misura1 = igrometroTerra.misura(lt, false);  
        lt = lt.plusMinutes(1); // rimisura dopo 1 minuto  
        float misura2 = igrometroTerra.misura(lt, true);  
        assertTrue(misura2 > misura1);  
    }  
  
    @Test  
    public void testMisura_Irr_OFF(){  
        IgrometroTerra igrometroTerra = new IgrometroTerra(0, null, 10);  
        LocalDateTime lt = LocalDateTime.now();  
        float misura1 = igrometroTerra.misura(lt, false);  
        lt = lt.plusMinutes(1); // rimisura dopo 1 minuto  
        float misura2 = igrometroTerra.misura(lt, false);  
        assertTrue(misura2 <= misura1);  
    }  
}
```

■ 3/3 - test DAOs

4.3 ORM Test

È stato inserito un test che riguarda la classe `OrdineDAO`, nel package `ORM`.

Questa classe, che si occupa della gestione degli Ordini nel database, invoca un'altra classe "DAO" al suo interno, in quanto ha la responsabilità di inizializzare l'oggetto della classe `Ordine`, che contiene l'attributo `piante`, una `arraylist` di `Piante`.

Quindi, come visibile nello [Snippet 18](#), questo test inserisce un ordine (`ordineRichiesto`) nel database tramite `OrdineDAO` e le relative piante con `PiantaDAO`; poi chiama il metodo `get` di `OrdineDAO` e verifica che i due ordini corrispondano.

In particolare verifica che le piante contenute nei due ordini siano dello stesso tipo e in quantità uguale (attraverso dei "contatori").

Se il test è superato, `OrdineDAO` ha inizializzato correttamente le piante.

- + test Queries (if not covered by functional tests on the business logic)

```
@Test
public void testInitPianteOrdineDAO() {
    System.out.println("--Test Inizializzazione Piante con richiesta a OrdineDAO--");
    System.out.println("Verifica che OrdineDAO, tramite PiantaDAO, inizializzi la lista di piante");

    ArrayList<Ordine> ordini = ordineDAO.get(Map.of("id", ordineRichiesto.getId()));
    assertFalse(ordini.isEmpty()); // non deve essere vuoto

    Ordine ordineOttenuto = ordini.get(0);
    assertNotNull(ordineOttenuto); // non deve essere null

    // verifico che la quantità di piante totali sia uguale
    assertEquals(ordineOttenuto.getnPiante(), ordineRichiesto.getnPiante());

    // creo una mappa che conta le piante di ogni tipo RICHIESTE
    Map<String, Integer> counterRichieste = new HashMap<>();
    for (Pianta p : ordineRichiesto.getPiante()) {
        if (counterRichieste.containsKey(p.getTipoPianta()))
            counterRichieste.put(p.getTipoPianta(), counterRichieste.get(p.getTipoPianta()) + 1);
        else
            counterRichieste.put(p.getTipoPianta(), 1);
    }

    // creo una mappa che conta le piante di ogni tipo OTTENUTE
    Map<String, Integer> counterOttenute = new HashMap<>();
    for (Pianta p : ordineOttenuto.getPiante()) {
        if (counterOttenute.containsKey(p.getTipoPianta()))
            counterOttenute.put(p.getTipoPianta(), counterOttenute.get(p.getTipoPianta()) + 1);
        else
            counterOttenute.put(p.getTipoPianta(), 1);
    }

    // verifico che le quantità di piante siano uguali
    for (String key : counterOttenute.keySet()) {
        assertEquals(counterOttenute.get(key), counterRichieste.get(key));
    }
    System.out.println("Test superato!");
}
```

- Document the database design with an Entity Relationships Diagram

- ... and a relational model

Admin (id, email, password, nome, cognome)

Operatore (id, email, password, nome, cognome, working, ruolo)

Cliente (id, email, password, nome, cognome)

Ordine (id, cliente, dataConsegna, piante, totale, stato)

Pianta (id, tipo, descrizione, dataInizio, costo, stato, ordine)

Spazio (id)

Settore (id, spazio_id, termometro, igrometroAria, fotosensore, lampada, climatizzatore)

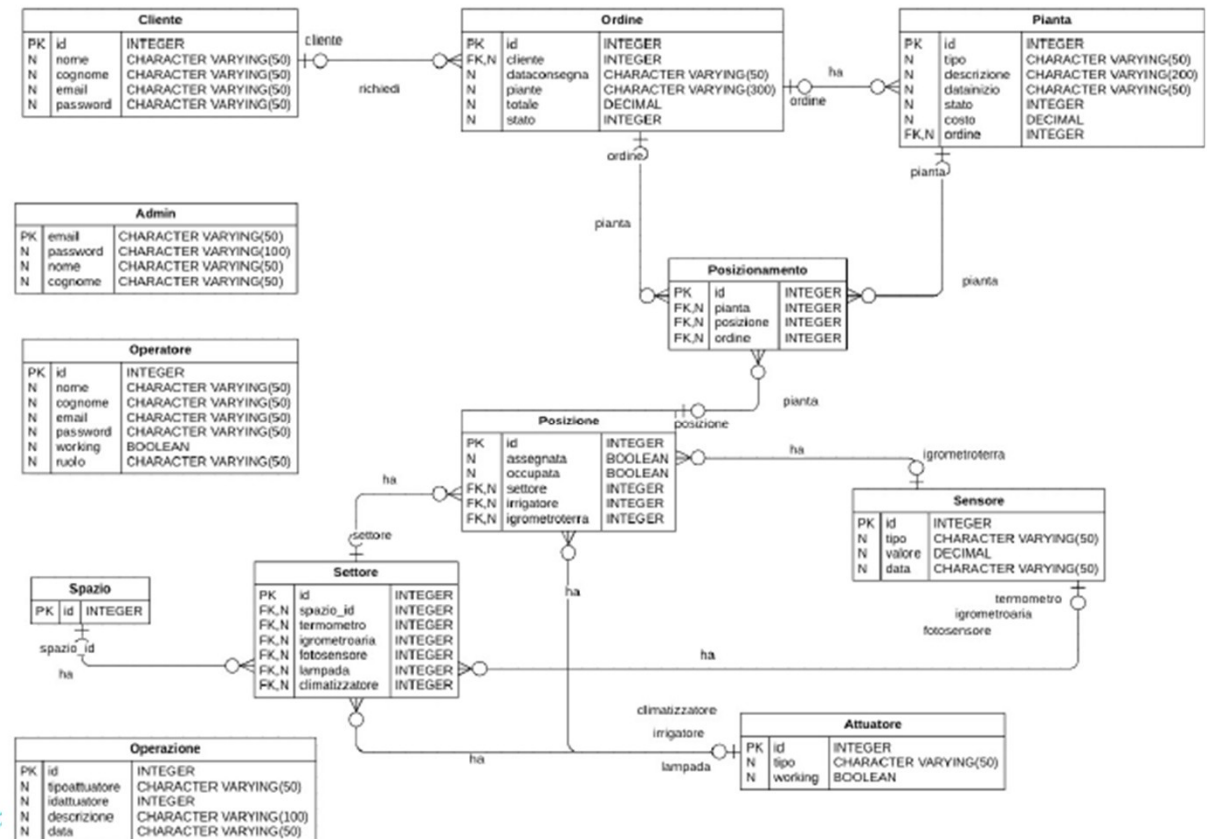
Posizione (id, settore, occupata, assegnata, irrigatore, igrometroTerra)

Sensore (id, tipo, valore, data)

Attuatore (id, tipo, working)

Posizionamento (id, pianta, posizione, ordine)

Operazione (id, tipoAttuatore, idAttuatore, descrizione, data)



- Represent alternative courses
- Associate mockups (or screenshots) to template steps
- + give each UC a number
- + login is not really a User Goal

UC	Login
Level	User Goal
Actor	Passeggero/Admin
Basic Course	1. L'utente accede al sistema. 2. IL sistema mostra la finestra iniziale dell'applicazione (Figura 3). 3. L'utente inserisce i propri credenziali per effettuare il login: username e password. 4. L'utente clicca su <i>Login</i>. 5. Il sistema mostra l'interfaccia Admin o Passenger a seconda dei credenziali di accesso inseriti (Figura 4 e 10).
Alternative Course	1. Il sistema non trova nessuna corrispondenza con i credenziali inseriti perché l'utente non si è ancora autenticato presso il sistema. (a) Il sistema mostra un messaggio di errore. (b) L'utente inserisce un username univoco al sistema. (c) L'utente inserisce una password che rispetta i vincoli imposti. (d) L'utente inserisce un'email che rispetta il formato standard. (e) L'utente clicca su <i>Register</i>. 2. Il sistema non trova nessuna corrispondenza con i credenziali inseriti perché l'utente ha inserito scorrettamente l'username o la password. (a) Il sistema mostra un messaggio di errore. (b) Viene fornita all'utente la possibilità di inserire nuovamente i credenziali.

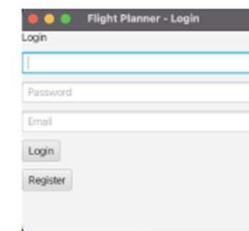


Figura 3: Finestra iniziale

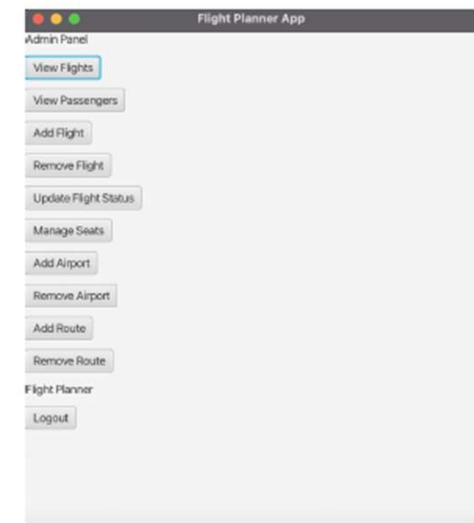


Figura 4: Finestra che mostra l'interfaccia di Admin con le sue funzionalità riservate

- represent alternative courses
- + represent pre- and post-conditions
- + consider covering basic and alternative courses with tests, identified by the UC number

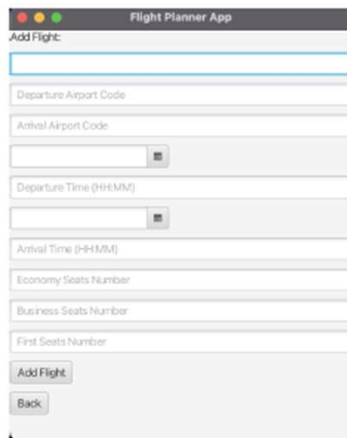
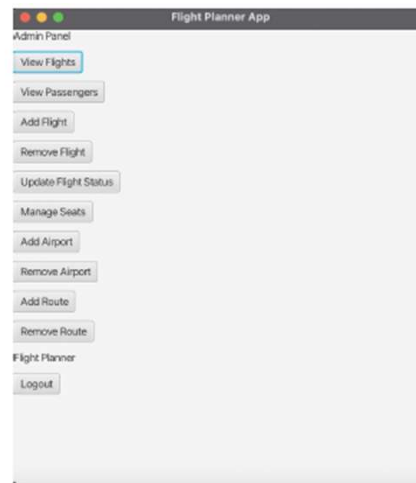



Figura 4: Finestra che mostra l'interfaccia di Admin con le sue funzionalità riservate

Figura 6: Finestra che mostra Add Flight e i campi necessari da completare per aggiungere un nuovo volo

UC	Add Flight
Level	Admin Goal
Actor	Admin
Basic Course	1. Dopo aver fatto accesso al sistema, l'amministratore clicca su <i>Add Flight</i>. 2. Il sistema mostra una finestra per l'inserimento dei dati del nuovo volo da aggiungere (Figura 6). 3. L'amministratore inserisce il codice univoco del volo. 4. L'amministratore inserisce codici univoci validi per gli aeroporti di partenza e di arrivo, che corrispondono a una rotta esistente. 5. L'amministratore seleziona la data di partenza. 6. L'amministratore inserisce l'ora di partenza secondo il formato richiesto. 7. L'amministratore ripete gli stessi passaggi per la selezione della data e l'inserimento dell'ora di arrivo. 8. L'amministratore specifica la quantità di posti disponibili per ciascuna delle tre classi (Economy, Business, First) su un nuovo volo. 9. L'amministratore clicca su <i>Add Flight</i>. 10. Dopo aver effettuato l'aggiunta del nuovo volo, l'amministratore clicca su <i>Back</i>. 11. Il sistema mostra il menù principale dell'applicazione (Figura 4).

UC	Add Flight
Level	Admin Goal
Actor	Admin
Alternative Course	1. L'amministratore omette di compilare i campi obbligatori e clicca su <i>Add Flight</i>. (a) Il sistema mostra un messaggio di errore. (b) Viene fornita all'amministratore la possibilità di completare i campi mancanti. 2. L'amministratore inserisce un codice aeroportuale di partenza o di arrivo inesistente. (a) Il sistema mostra un messaggio di errore. (b) Viene fornita all'amministratore la possibilità di inserire nuovamente i codici aeroportuali. 3. L'amministratore inserisce dei codici aeroportuali validi, ma non esiste una rotta corrispondente. (a) Il sistema mostra un messaggio di errore. (b) Viene fornita all'amministratore la possibilità di inserire nuovamente i codici aeroportuali. 4. L'amministratore non seleziona la data di partenza o di arrivo. (a) Il sistema mostra un messaggio di errore. (b) Viene fornita all'amministratore la possibilità di selezionare le date che sono ancora mancanti. 5. L'amministratore non inserisce l'ora di partenza o l'ora di arrivo nel formato richiesto. (a) Il sistema mostra un messaggio di errore. (b) Viene fornita all'amministratore la possibilità di inserire nuovamente le ore di partenza o di arrivo. 6. L'amministratore non inserisce la quantità disponibile dei posti in formato numerica. (a) Il sistema mostra un messaggio di errore. (b) Viene fornita all'amministratore di inserire nuovamente la quantità dei posti disponibili. 7. L'amministratore inserisce un numero totale di posti inferiore a 1. (a) Il sistema mostra un messaggio di errore. (b) Viene fornita all'amministratore di inserire nuovamente la quantità dei posti disponibili.

- Created with any tool where editing requires negligible time with respect to conceptual design
- Design focused on information content and controls in the page
- (replaced by screenshots if an interface is developed)



- Emphasis on the routing across pages

- Package diagram, accompanied by a discussion of responsibilities and type of classes in each package, and dependencies, and any other remark useful to convey the concept of design
- Class diagram of each package, accompanied by a description ... shall not be like a «prose della poesia» rather it shall support interpretation of the diagram with a set of remarks



Document the architecture

- Give each title a meaning
to obtain a self-explaining index

Indice

1	Analisi	4
1.1	Statement	4
1.2	Architettura	5
1.3	Strumenti Utilizzati	6
2	Progettazione	6
2.1	Use Case Diagram	6
2.2	Use Case Template	8
2.3	Page navigation diagram	10
2.4	View	11
2.5	Mockup	12
2.6	Database	15
2.7	Struttura delle classi	17
2.7.1	Domain Model	17
2.7.2	Controller	18
2.7.3	Business Logic	19
2.7.4	ORM	20
3	Implementazione	21
3.1	Domain model	22
3.1.1	BaseUser	22
3.1.2	Customer	22
3.1.3	Personal Trainer	22
3.1.4	Appointment	22
3.1.5	Course	22
3.1.6	Exercise	22
3.1.7	Machine	22
3.1.8	Schedule	22
3.1.9	Signed	22
3.1.10	Training	22
3.2	Business logic	23
3.2.1	NavigationManager	23
3.2.2	ServiceManager	23
3.2.3	ServiceFactory	23
3.2.4	BaseUserService	23
3.2.5	ScheduleService	23
3.2.6	ExerciseDetailService	23
3.2.7	ExerciseService	23
3.2.8	TrainingService	24
3.3	Business logic Customer	24
3.3.1	BookAppointmentService	24
3.3.2	BookCourseService	24
3.3.3	ProfileService	24
3.4	Business logic Personal Trainer	24
3.4.1	AgendaService	24
3.4.2	MachineService	24
3.4.3	ProfilePTService	24
3.5	DAO	25
3.5.1	CustomerDAO	25
3.5.2	PersonalTrainerDAO	25
3.5.3	AppointmentDAO	25
3.5.4	CourseDAO	25
3.5.5	ExerciseDAO	25
3.5.6	ExerciseDetailDAO	25
3.5.7	MachineDAO	25
3.5.8	ScheduleDAO	26
3.5.9	SignedDAO	26
3.5.10	TrainingDAO	26
4	Test	27
4.1	Tipologie di test effettuati e organizzazione	27
4.2	Test di integrazione	28
4.2.1	AppointmentDAOTest	28
4.2.2	CourseDAOTest	28
4.2.3	CustomerDAOTest	29
4.2.4	ExerciseDAOTest	29
4.2.5	ExerciseDetailDAOTest	30
4.2.6	MachineDAOTest	30
4.2.7	PersonalTrainerDAOTest	30
4.2.8	ScheduleDAOTest	31
4.2.9	SignedDAOTest	31
4.2.10	TrainingDAOTest	31
4.3	Test funzionali	32
4.3.1	BookCourseServiceTest	32
4.3.2	BookAppointmentServiceTest	32

- Provide a clear yet synthetic statement of the application context and aim

1.1 Statement

TrainMate è un'applicazione che permette ai clienti che ne fanno uso di tenere traccia della propria attività fisica. L'applicazione dà la possibilità di gestire le singole sessioni di allenamento offrendo una visione generale del proprio andamento con gli obiettivi raggiunti e quelli da raggiungere. Lo scopo è quello di permettere una maggiore qualità ed efficacia di allenamento, attraverso la creazione di schede personalizzate.

- **Gestione e visualizzazione dell'allenamento:** Utilizzando TrainMate, il Customer avrà la possibilità di tenere traccia degli allenamenti svolti e di aggiornarne i dettagli della propria scheda in base ai progressi fatti.
- **Gestione prenotazioni e corsi:** L'applicazione mostra al cliente, una volta effettuato il login, i vari corsi di fitness proposti dalla palestra, i quali possono essere frequentati in aggiunta agli allenamenti fatti con la scheda fornitagli dal Personal Trainer. Dal punto di vista del Personal Trainer, quest'ultimo può scegliere gli orari dei propri corsi e, in base a questi, gestire gli appuntamenti di allenamenti privati richiesti da singoli clienti.
- **Visualizzazione agenda e Gestione macchinari:** L'applicazione dà al personal trainer la possibilità di visualizzare la propria agenda settimanale con relativi impegni e corsi programmati. Per quanto riguarda invece la parte di gestione dei macchinari, è presente una sezione in cui il personal trainer può abilitare o disabilitare ed aggiungere o eliminare dei macchinari.
- **Creazione e gestione del profilo personale:** In questa sezione dell'app il cliente potrà inserire le proprie caratteristiche fisiche come altezza, peso, età, sesso e i suoi obiettivi (aumentare massa muscolare, perdere peso...). In base ai dati inseriti, il Personal Trainer potrà creare una scheda. Quest'ultima sarà suddivisa in sezioni ed ognuna di queste conterrà una serie di esercizi che potranno essere svolti in allenamento. Per quanto riguarda la gestione della sezione, verrà data al cliente la possibilità di aggiornare i propri dati ed i propri obiettivi; in corrispondenza dei cambiamenti personali, il cliente potrà richiedere un appuntamento con il PT per l'aggiornamento della scheda.

- Document the architecture with packages and their dependencies
- Add a textual description of responsibilities and constituency of each package

1.2 Architettura

L'architettura del progetto è suddivisa in diversi package per garantire la specializzazione dei compiti tra le diverse parti. Il package **View** rappresenta l'interfaccia tra l'utente ed il sistema. Il **Controller** gestisce le richieste dell'utente, tramite l'utilizzo della classe **Engine**. Il package **Business-Logic/Service** contiene le classi che gestiscono la logica del programma, le quali sono collegate al Domain Model e alle classi DAO del package **ORM**. Quest ultimo si occupa della comunicazione con il database tramite una connessione stabilita tramite **JDBC** in maniera tale da tenere sempre aggiornate le classi del Model e della BusinessLogic. Questa struttura permette una gestione modulare delle operazioni del sistema, garantendo una separazione tra la logica di presentazione, la logica di controllo e la logica di business.

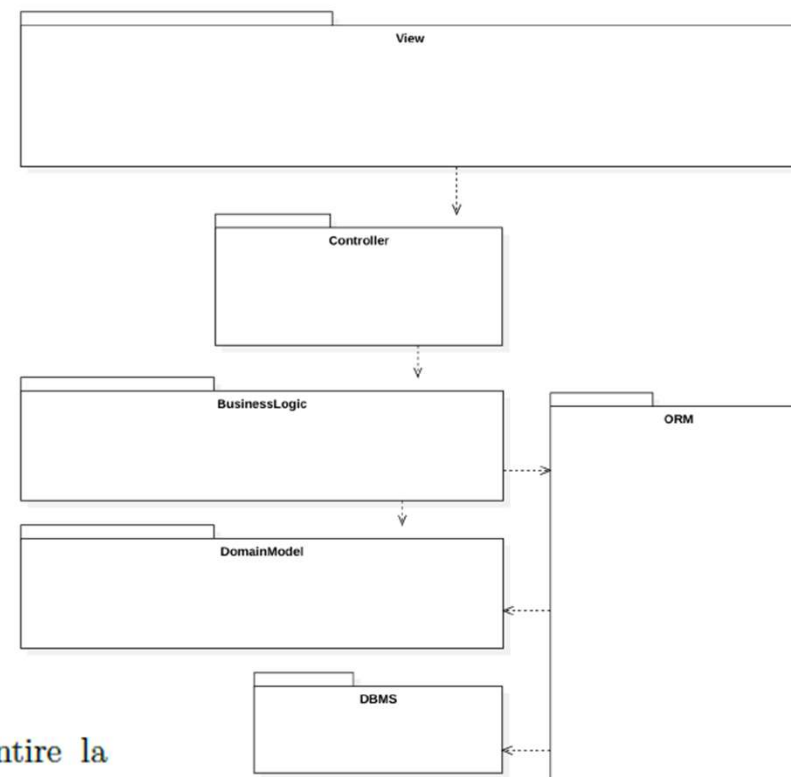
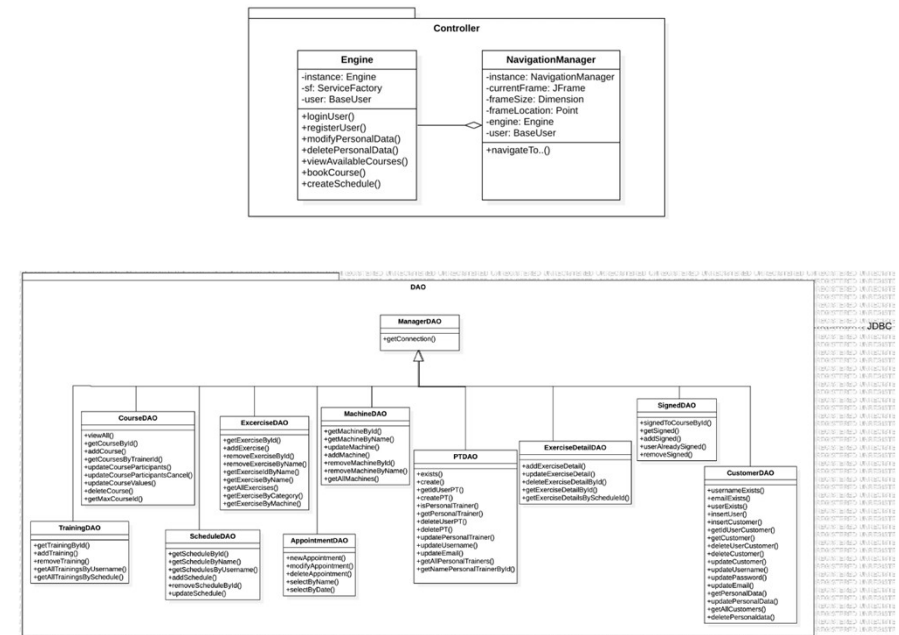
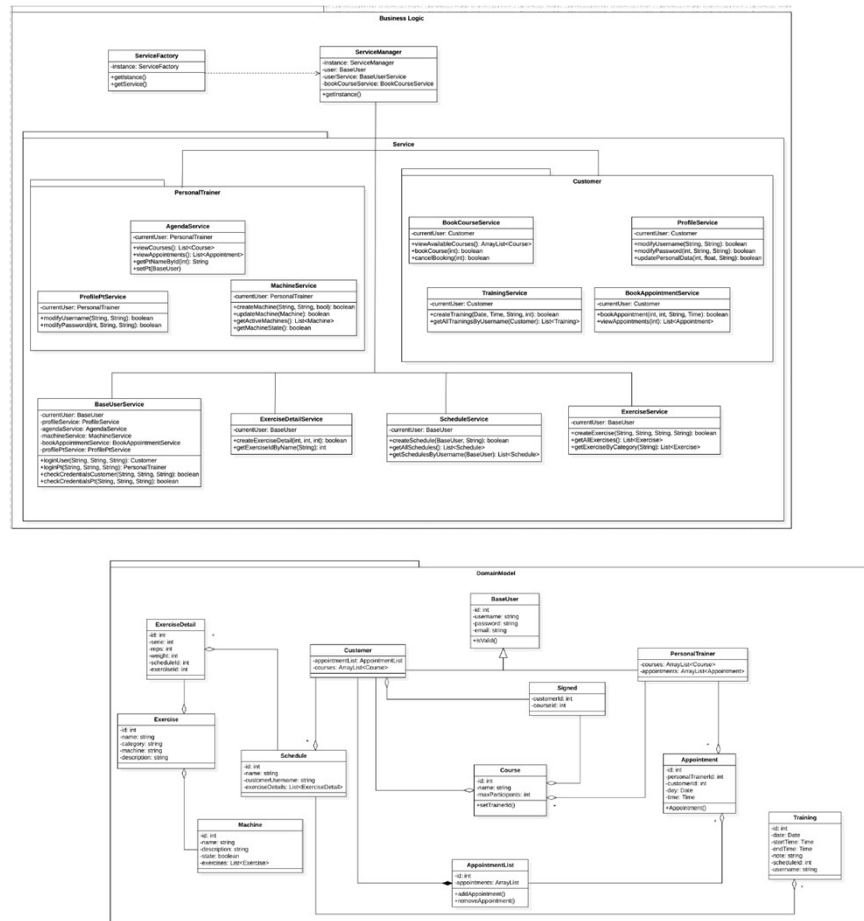


Figure 1: Descrizione dei package utilizzati

- Each package will later on be expanded into classes in the section documenting detailed design



■ Document used technologies and tools

1.3 Strumenti Utilizzati

- StarUML per la creazione dei diagrammi
- IntelliJ IDE per lo sviluppo del software
- MySQL Workbench per la creazione e gestione del database
- Visily.ai per la creazione dei mockup
- Java Swing per la realizzazione delle View

■ Associate template steps with available mockups

UC 1	Registrazione
Level	User goal
Actor	Customer, Personal Trainer
Basic course	1. L'utente entra nella pagina di registrazione (Figure 7) 2. Inserisce le proprie credenziali (se il PT inserisce anche la chiave di registrazione) 3. Il sistema verifica la validità dei dati inseriti 4. Il sistema registra il nuovo utente
Alternative course	3a. L'utente inserisce dati non validi ed il sistema concede un'altra possibilità

UC 4	Crea nuova scheda
Level	User goal
Actor	Personal Trainer
Basic course	1. Il personal trainer dalla homepage può accedere alla sezione di aggiunta scheda 2. Il sistema reindirizza alla pagina richiesta 3. Il personal trainer può nominare la scheda ed assegnarla ad un cliente 4. Dopodiché può andare nella sezione dei dettagli e modificare tutti i dettagli degli esercizi nella maniera più adatta al cliente (Figure 10).

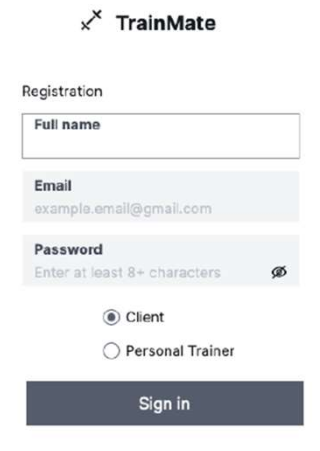


Figure 7: Mockup Registrazione utente

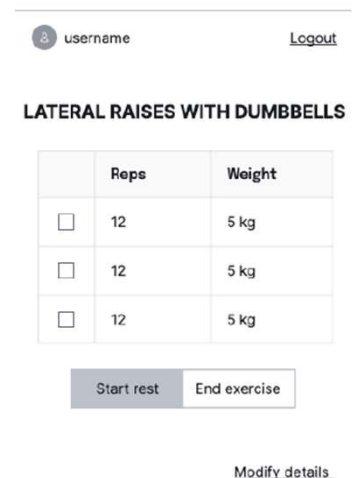


Figure 10: Mockup Specifiche allenamento

- Consider decomposing the use case diagrams for different roles

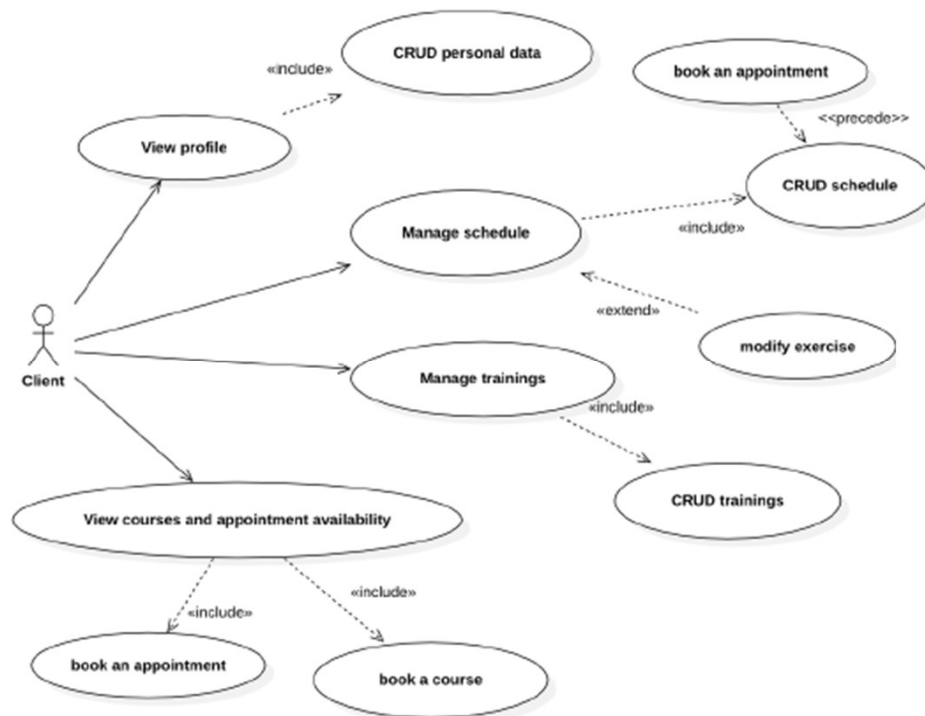


Figure 3: diagramma Use Case per il customer

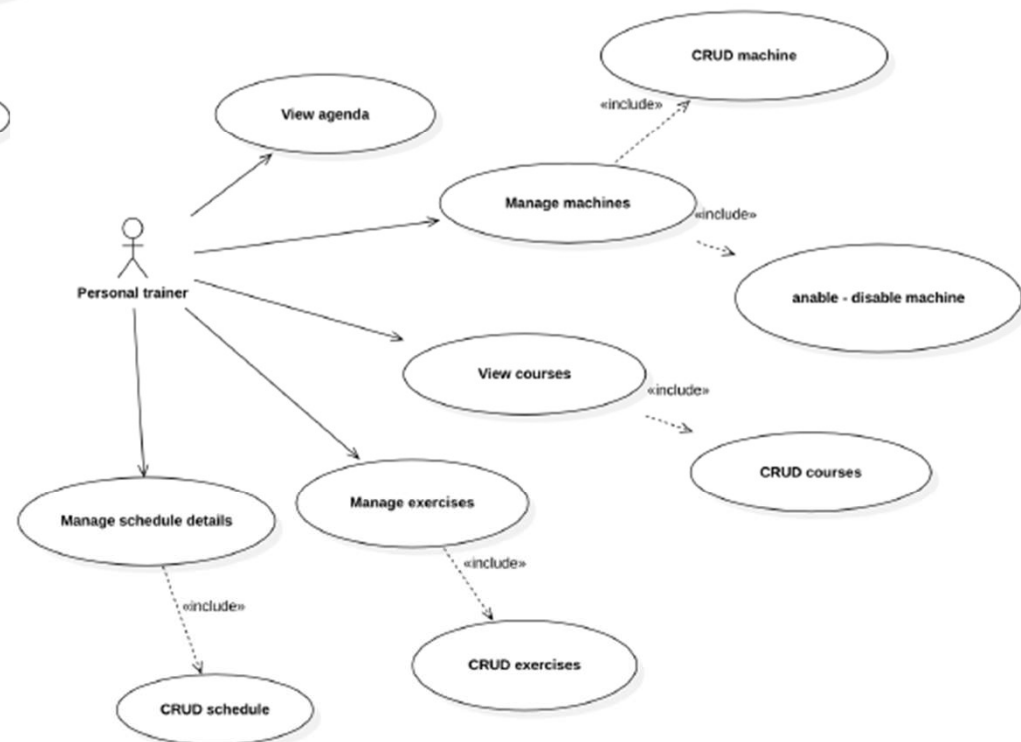
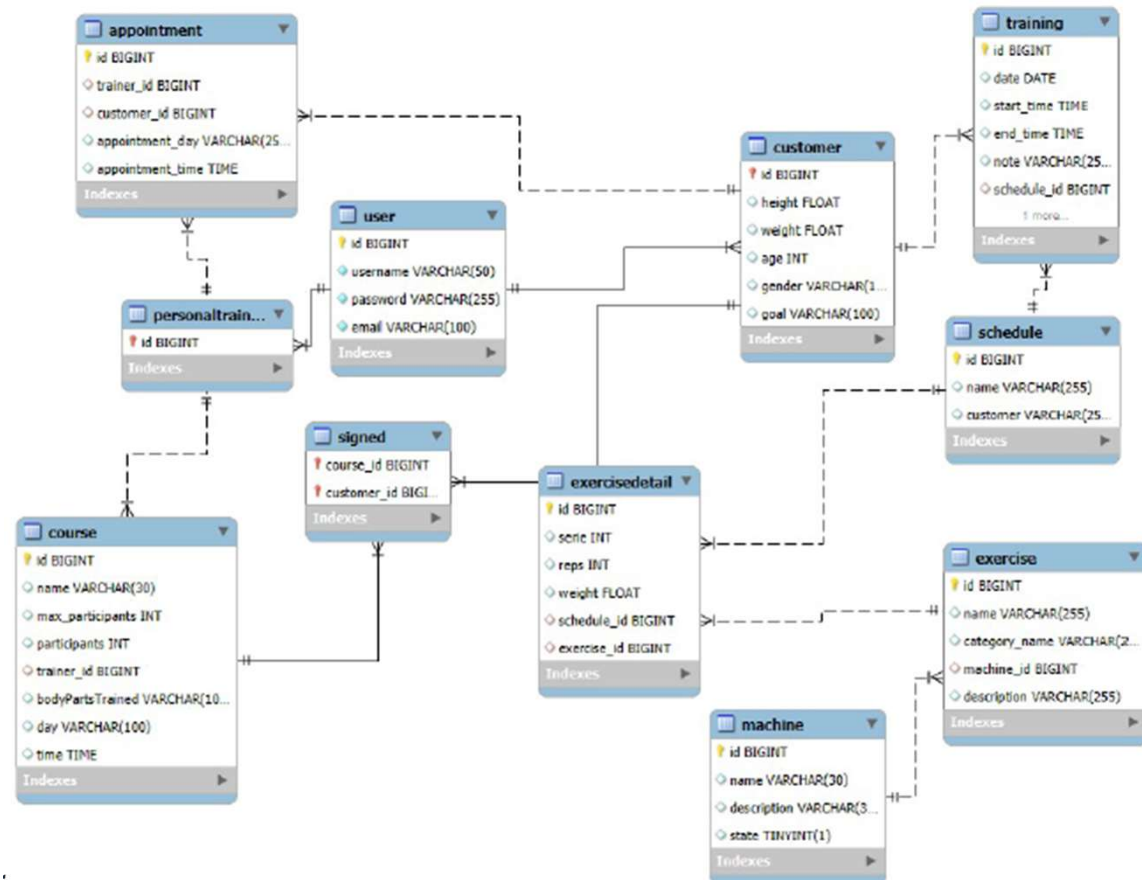


Figure 4: diagramma Use Case per il personal trainer

- Number alternative courses
with a fork from the step where they originate

UC 3	Prenota un corso
Level	User goal
Actor	Customer
Basic course	1. Il Customer dalla sezione Visualizza corsi può scegliere di prenotarsi ad un corso 2. Il sistema reindirizza alla pagina contenente l'elenco dei corsi programmati 3. Il Customer sceglie un corso e si prenota alla lezione
Alternative course	3a. Se è già stata raggiunta la capienza massima per quell'orario di lezione, il cliente può prenotarsi per un orario differente tra quelli proposti per il corso scelto

- Document the database design with an Entity Relationships Diagram



TBD: a recurrent question: primo

- Se lavori con JDBC, entrambi gli approcci hanno un senso ... con tecnologie di livello enterprise, non ci sono dubbi, si parte dalle classi (a meno di avere vincoli di legacy)



- Associate steps of courses with mockups

- The page navigation diagram can be repeated for different user types

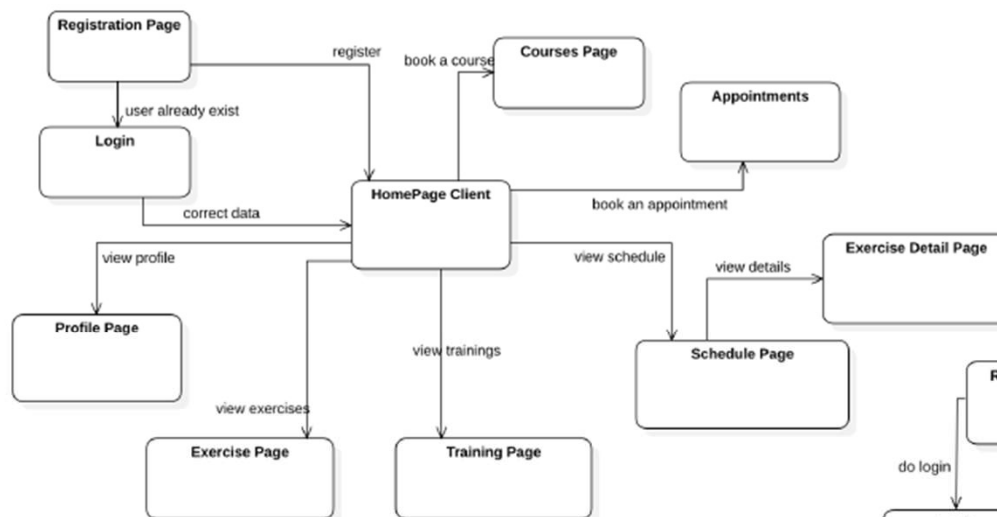


Figure 5: Page navigation diagram Customer

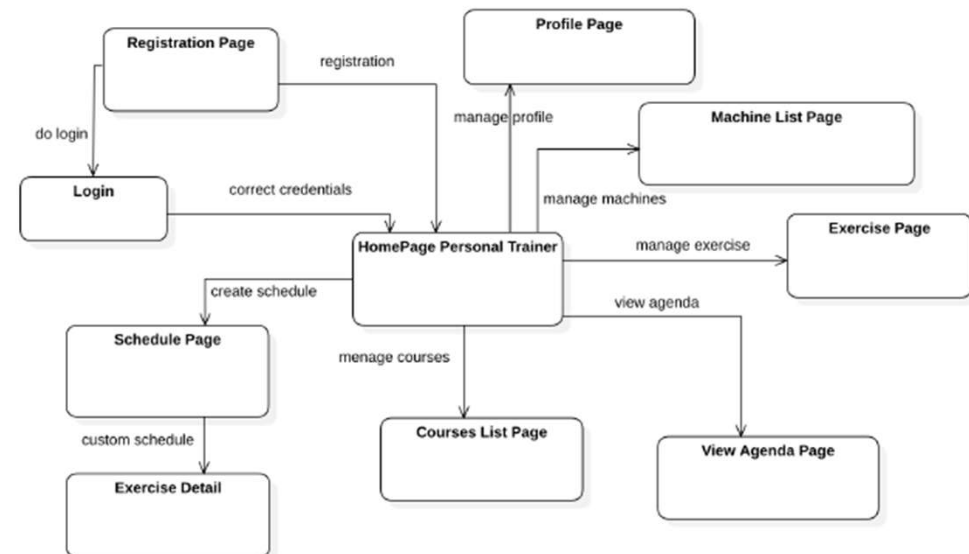


Figure 6: Page navigation diagram personal trainer